Cloud providers, from AWS to Azure to Google Cloud, as well as observability vendors, from Datadog to Splunk to New Relic to Lightstep to Dynatrace to Honeycomb to Grafana Labs, have decided to assemble behind OpenTelemetry. This effectively means the telemetry industry has decided to make telemetry table stakes, to commoditize it, and, instead, compete on the destinations (storage and query as well as frontends).

OpenTelemetry covers, in order of maturity and at the time of writing, traces, metrics, and logs. Going forward, the community has decided to also cover profiles (resource usage and timing within a process) in OpenTelemetry. See also our discussion in chapter 2.

Since logs are still a work in progress in 2022, I will use vendor-specific or signal-specific setups to demonstrate logs in this chapter. For metrics and traces, on the other hand, I will show you how to benefit from the open and vendor-agnostic standard that OpenTelemetry and its agent, the collector, is.

4.1 Log routers

First, we will have a look at log routers. These agents allow you to collect logs from various sources, be that on a system level or your applications, and send them to a central place for lookup. In this context, you often hear also about log forwarding, log collection, and log aggregation. However, to keep things simple, we will gloss over the details and lump all these together as *log routers*.

At time of writing (2023), vendor-specific agents, such as Splunk's Universal Forwarder or the Datadog agent, as well as signal-specific agents, including Fluentd/Fluent Bit or the widely used Logstash. Expect that to change and move to OpenTelemetry, going forward.

We will differentiate between system logs and application logs. System logs come from, but are not limited to, the following sources (refer to our discussions in chapter 2 for details):

- The operating system (e.g., in Linux, from `/var/log` and `journalctl`)
- The Kubernetes control plane (http://mng.bz/BmWv) logs (including from the API server, etcd, and controller manager) and the worker node logs (http://mng.bz/d10D), such as from the kubelet or the container runtime
- User activity and API usage logs, like the ones provided by AWS CloudTrail (https://aws.amazon.com/cloudtrail/)
- Infrastructure automation logs (e.g., from HashiCorp Terraform or AWS CloudFormation)

Application logs, on the other hand, are what you emit in your code. You may end up using different agents for system logs and application logs; however, if possible, you should try to standardize by choosing one, for example, from the ones discussed in the following sections. This is mostly for operational reasons, as it means there will be less to patch (think of the Log4j challenges, for example) and upgrade.